Victor Roberto Curzio

Full-Stack Engineer · Node.js · TypeScript · Backend Architecture · AI-Native Development

La Plata, Argentina · victor.curzio@hotmail.com · +54 221 542 4585
linkedin.com/in/victor-roberto-curzio · github.com/VicCurzio · viccurzio.github.io/portfolio

SUMMARY

Three years building backend systems in production with Node.js and TypeScript, focused on system design and data integrity. At Grupo DELSUD I am the technical reference for a team of 5: I designed a 20-module management system from scratch, connected it to a ~25-model CRM already in production and launched it with no cutover window. Fifteen people operate it today over more than 200 contracts, and I brought its heaviest queries down from 3 seconds to 0.6 through query optimization. In parallel I am tech lead at 2winGs, an early-stage B2B product where I own the architecture: I defined the system design and the build order before writing any code, on NestJS with Fastify, PostgreSQL, Redis queues and automated CI/CD. On a personal project I built a question-answering system over private documentation (RAG on PostgreSQL and pgvector) covering 340 notes and 5,338 chunks, with an automated 34-question evaluation that scores 89% correct answers and 100% correct refusals when the answer is not in the documents. I work every day with agentic coding tools, with the specification written before the code.

EXPERIENCE

Full-Stack Developer · Technical Reference — Grupo DELSUD

Jan 2026 — Present · La Plata, Argentina

- 20 functional modules (contracts, collections, cash flow, inflation indexing, reporting): I designed the architecture of an administrative and accounting management system from scratch and took it to production. Fifteen people across sales, administration and collections operate it today, over more than 200 contracts.
- From 3 seconds to 0.6 on the system's heaviest queries, five times faster: query optimization and indexing driven by measurement, not by intuition.
- Two databases in one product: I connected the existing CRM (Sequelize, ~25 models) with the new system (Drizzle ORM) through a dual-database architecture with automatic two-way synchronisation, and no cutover window for users.
- Twenty people across 3 departments use the workday time-tracking module I built end to end: 14 REST endpoints and an immutable append-only log that nobody can edit after the fact, with role-based permissions.
- Technical reference for a team of 5 (2 devs, UX, QA, PM): I set the code standards, review changes and take part in the key technical decisions together with the tech lead.
- Microservices architecture in the group's internal ERP: one Node/PostgreSQL service per department, with real-time notifications over WebSocket between them.
- Fixed critical data-integrity bugs, including race conditions on concurrent record creation and on the synchronisation between the two systems.
- Access control (RBAC): token verification and role authorisation middlewares protecting every endpoint according to the user profile.

Stack: Node.js · TypeScript · Express.js · NestJS · PostgreSQL · MySQL · Drizzle · Sequelize · Socket.io · React

Full-Stack Developer and Tech Lead (Freelance) — 2winGs International Group LLC

Aug 2026 — Present · Remote

- Early-stage product: I defined the system design and the build order, from the data model to the REST API contract, written down as technical decisions before writing a line of code.
- REST API in NestJS on Fastify with Drizzle and PostgreSQL, and heavy work moved to background jobs with BullMQ and Redis (run in Docker) so the user's request does not wait for it.
- CI/CD on GitHub Actions: it builds off the server, copies the artifact over SSH, runs the migrations and only then restarts the process, with nginx as a reverse proxy in front of the API and two separate environments over HTTPS.
- Daily database backup to a provider other than the one hosting the server, restored against a throwaway database to prove it works: a backup nobody has restored is not a backup.
- Workflow built around agentic coding tools: the specification lives in the repository as the source of truth, and custom automated checks reject any change that does not meet the repo's standards.
- Privacy rules enforced in the data layer instead of the interface: a company reaches a full profile only for someone who applied to its opening.
- Plans, prices, limits and permissions live in the database, with no hard-coded values, so a commercial change is not a deployment.
- Every feature ships with automated tests: unit, integration and end-to-end (Playwright against the build), written against the class of error they guard and seen failing before being trusted.

Stack: TypeScript · NestJS · Fastify · PostgreSQL · Drizzle · Redis · BullMQ · Docker · nginx · PM2 · GitHub Actions · Playwright · Cloudflare R2

Full-Stack Developer — **Cognitive Link — Consulting & IT Solutions**

Mar 2025 — May 2026 · La Plata, Argentina

- 5 businesses running in parallel on a single instance: I built a multi-tenant SaaS platform from scratch with Next.js, TypeScript and Supabase, with more than 24 tables, per-client data isolation and role-based access control.
- Integrations with the Mercado Pago SDK and the Google Calendar API (OAuth 2.0) to automate appointment and payment handling.
- Scheduled jobs to send notifications and reminders automatically.

Stack: Next.js 14 · Node.js · TypeScript · PostgreSQL · Supabase · Tailwind CSS

Full-Stack Developer (Freelance) — **Cicaré**

Mar 2025 — Aug 2025 · Saladillo, Argentina

- Design and implementation of a scalable architecture with authentication and permission management for a platform holding confidential information.
- End-to-end full stack: persistence on Supabase, endpoints and administrative approval flows.
- Hierarchical content structure with advanced, customisable search filters.
- Automation of key notifications and interface work for faster navigation.

Stack: Next.js · Node.js · PostgreSQL · Supabase · Tailwind CSS

Full-Stack Developer — **Felanix Construcciones**

Sep 2023 — Sep 2024 · La Plata, Argentina

- Automated key processes of a construction management system that used to be manual, shortening task processing time.
- Developed the core business logic, applying Scrum to coordinate timelines and requirements.
- Managed timelines and requirements, keeping the project on schedule.
- Full technical support: hardware and software incidents to keep operations running.

Stack: Ruby on Rails · Active Record · PostgreSQL · Docker · JavaScript · Bootstrap · Trello

EDUCATION

Bachelor's Degree in Systems — Universidad Nacional de La Plata (UNLP) · Dec 2023 — Present

Courses

- Database Administration (MySQL) — Buenos Aires Ministry of Education (Nov. 2023)
- GNU/Linux Server Administration (Security) — Buenos Aires Ministry of Education (Dec. 2023)
- Professional Ruby on Rails Course — Código Facilito (Nov. 2023)
- Java OOP Bootcamp — Alkemy (Apr. 2024)
- React Front-end Development Track — Coderhouse (Feb.–Jul. 2023)
- Git and GitHub — Alura Latam (Apr. 2023)

SKILLS

Back-end
Node.js · TypeScript · Express.js · NestJS · Fastify · REST API design · WebSocket · Python

Architecture
System design · Microservices · Dual-DB · Job queues · RBAC · Migrations · Clean Code

Infrastructure
CI/CD · Docker · Linux · nginx · PM2 · GitHub Actions · DigitalOcean · Cloudflare

Applied AI
Agentic coding tools · RAG · pgvector · Embeddings · Hybrid search · Retrieval evaluation · Anthropic SDK · Tool-using agents

Databases
PostgreSQL · Redis · Query optimization · MySQL · SQLite · Supabase · Drizzle · Sequelize

Testing & Tools
Unit / integration / end-to-end tests · Vitest · Playwright · Git · GitHub · Postman · DBeaver

LANGUAGES

Spanish — Native · **English** — Intermediate (B1) — daily technical reading and writing

SOFT SKILLS

Autonomy · Analytical thinking · Technical curiosity · Clear communication · Initiative